

5.Array

Syeda Jannatul Naim

Lecturer | Dept. of CSE

World University of Bangladesh (WUB)

March, 2025

SUBJECT PRESENTATION – ARRAY in C

- INTRODUCTION TO ARRAY
 - NEED FOR ARRAY TYPE
 - HOW TO UPDATE AN ARRAY
 - ACCESS ARRAY ELEMENT IN C
 - TYPES OF ARRAYS
 - ALGORITHM AND CODES
 - Linear Search Algorithm
 - Binary Search Algorithm
 - Matrix Transpose
 - Matrix Multiplication
- 

INTRODUCTION TO ARRAY

- Array is a collection of elements of the same type that are referenced by a common name.
 - Arrays are of Derived data type, which are derived from the primitive data type, basically consist of int / float.
 - All the elements of an array occupy a set of contiguous memory locations.
 - To create an array: define the data type int and specify the name of the array followed by square brackets [].
 - SYNTAX: `data_type array_name[size];`
- 

NEED FOR ARRAY TYPE

- Consider the following issue:
- "We have a list of 1000 students marks of an integer type. If using the basic data type (int), we will declare something like the following which will be a lengthy decleration.

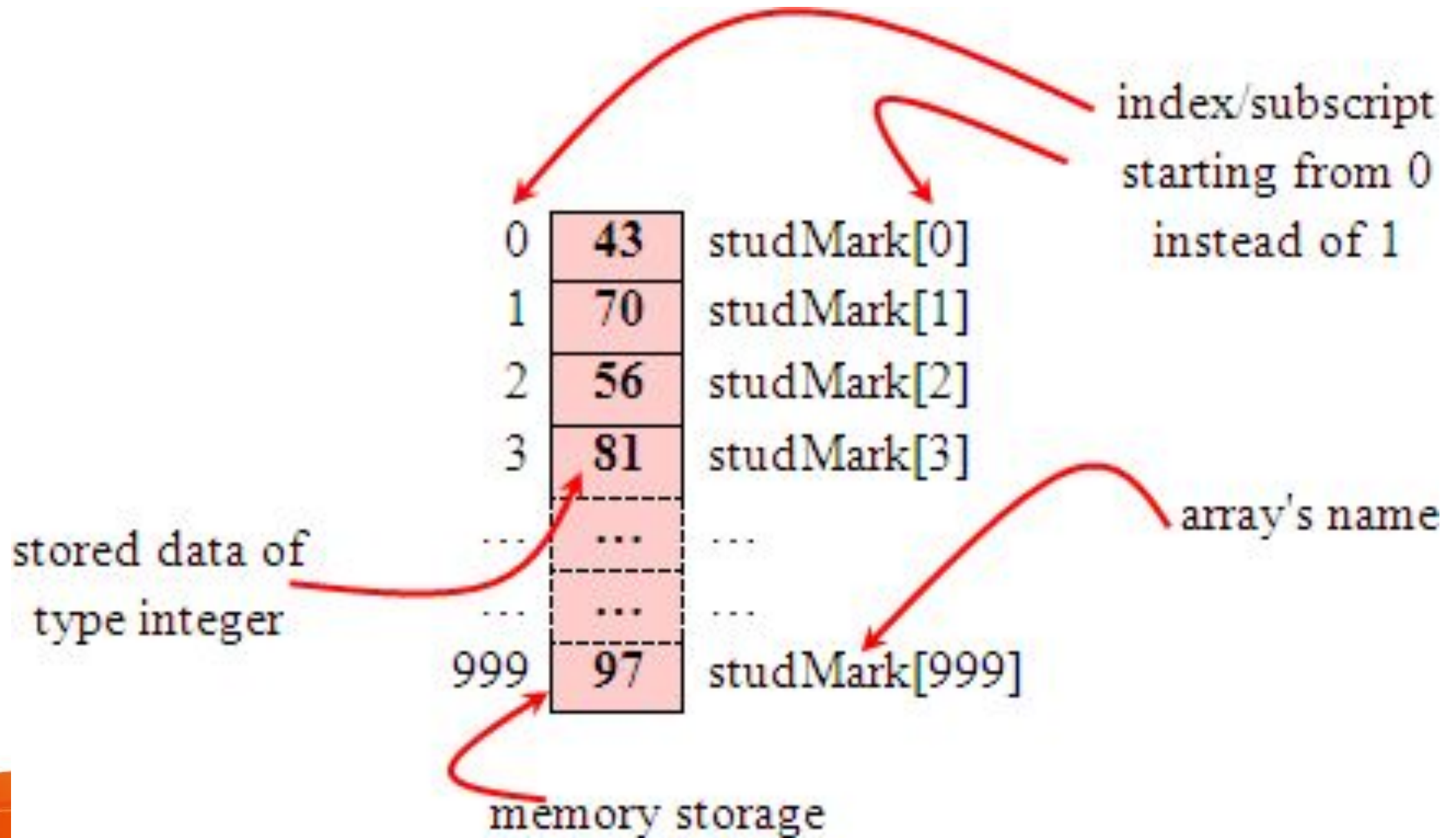
```
int studMark0, studMark1, .....studMark999
```

- By using an array, we just declare like this

```
int studMark[1000];
```

- This will reserve 1000 contiguous memory locations for storing the students marks.

Graphically, this can be depicted as in the figure.



HOW TO UPDATE AN ARRAY

- We can update the value of array elements at the given index *i* in a similar way to accessing an element by using the array square brackets `[]` and assignment operator `(=)`.
- SYNTAX: `array_name[i] = new_value;`
- The following is an example of Array updation

```
#include <stdio.h>
int main()
{
    int arr[5] = {2, 4, 8, 12, 16};
    // Update the first value of the array
    arr[0] = 1;
    printf("%d", arr[0]);
    return 0;
}
```

OUTPUT: 1

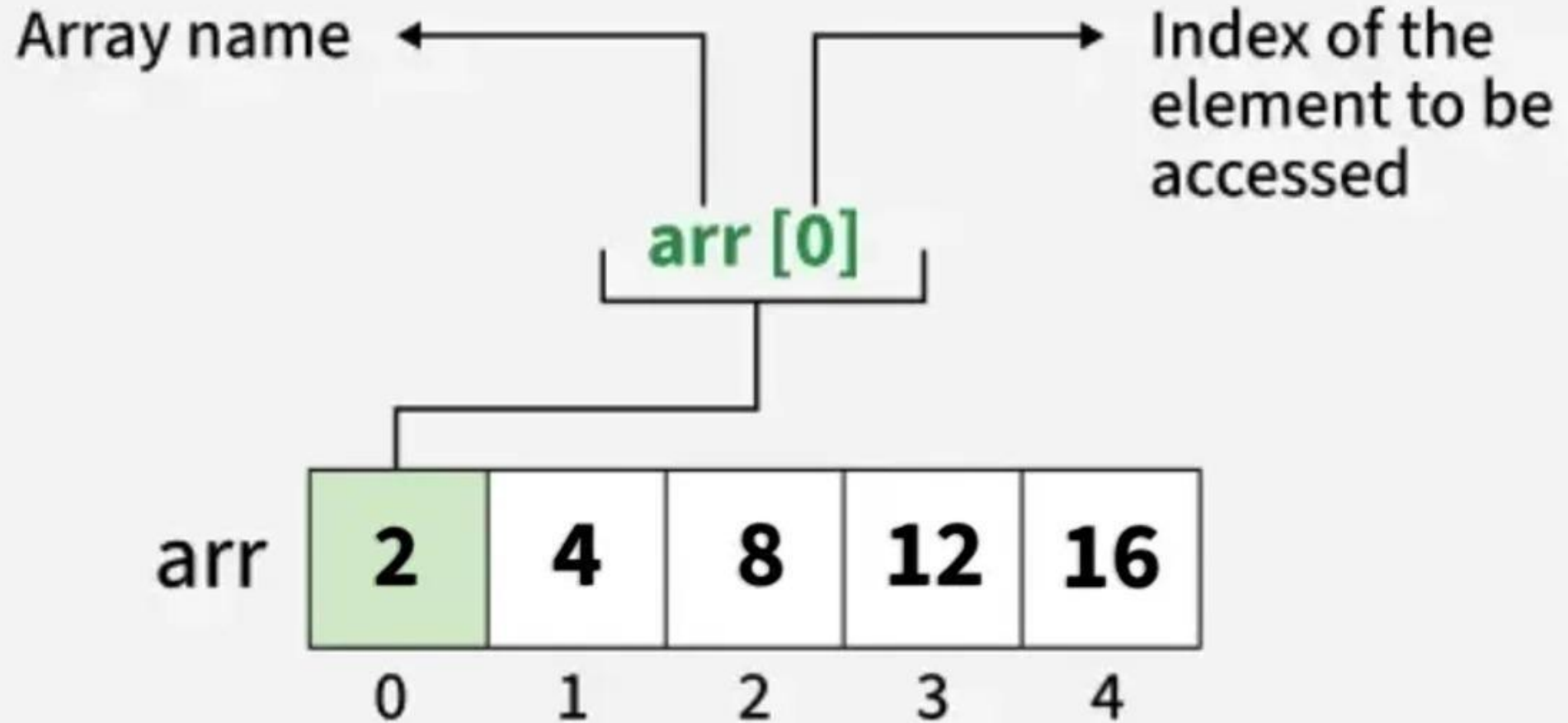


ACCESS ARRAY ELEMENT IN C

- To access an array element, refer to its index number.
- Array indexes start with 0: [0] is the first element. [1] is the second element, etc.
- This statement accesses the value of the first element [0] in myNumbers:
- Example

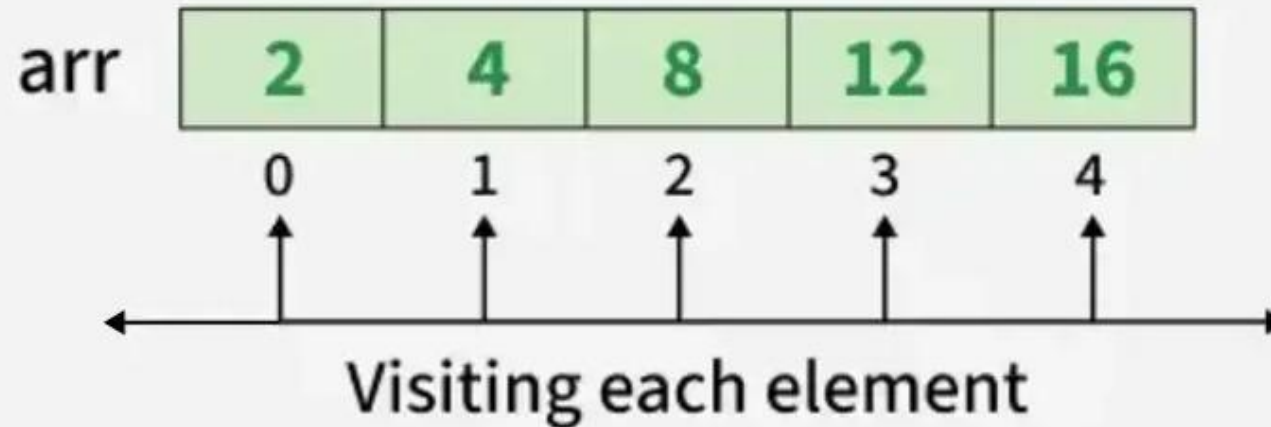
```
int arr[] = {2, 4, 8, 12, 16};  
printf("%d", arr[0]);
```


Output: 2



C ARRAY TRAVERSAL

- Array Traversal is the process in which we visit every element of the array in a specific order. For C array traversal, we use loops to iterate through each element of the array.



```
#include <stdio.h>
int main()
{
    int arr[5] = {2, 4, 8, 12, 16};
    printf("Printing Array Elements\n"); // Print each element of array using loop
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
    printf("Printing Array Elements in Reverse\n"); // Printing array element in reverse
    for(int i = 4; i >= 0; i--)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```



OUTPUT

Printing Array Elements

2 4 8 12 16

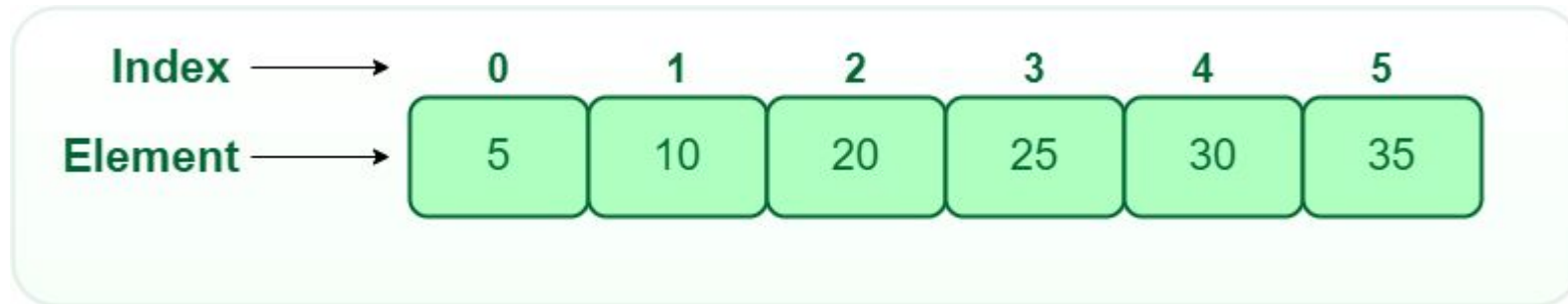
Printing Array Elements in Reverse

16 12 8 4 2



TYPES OF ARRAY

1. One-dimensional (1-D) array:



Syntax for Declaration of Single Dimensional Array

```
arraydata_type array_name[array_size];
```

where,

data_type: is a type of data of each array block.

array_name: is the name of the array using which we can refer to it.

array_size: is the number of blocks of memory array going to have.

For Example

```
int nums[5];
```

2. Two-dimensional (2D) array:

Syntax for Declaration of Two Dimensional Array

data_type array_name[sizeof_1st_dimension][sizeof_2nd_dimension];

where,

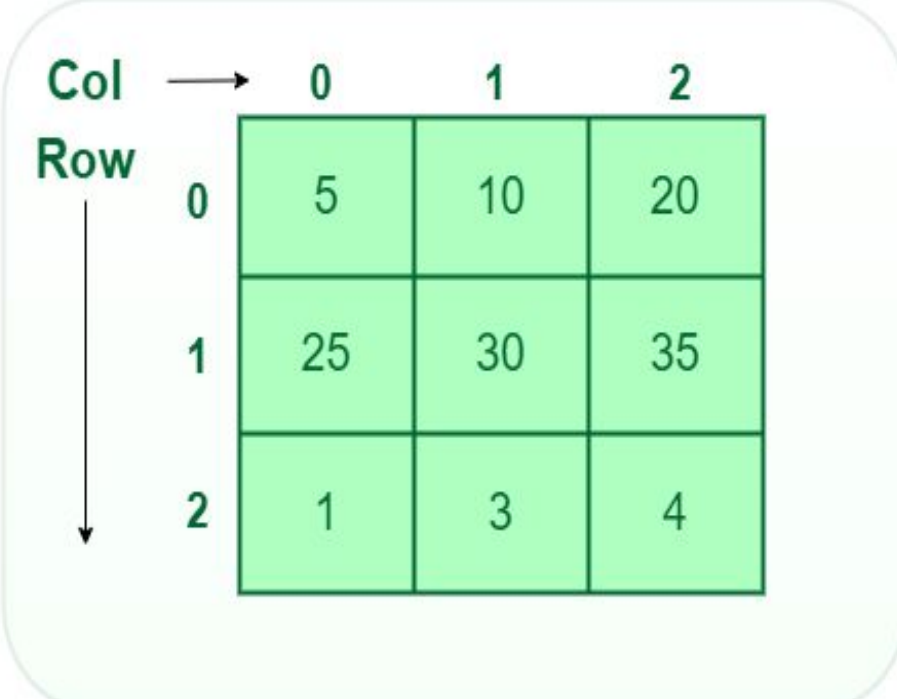
data_type: is a type of data of each array block.

array_name: is the name of the array using which we can refer to it.

sizeof_dimension: is the number of blocks of memory array going to have in the corresponding dimension.

For Example

int nums[5][10];



The diagram illustrates a 3x3 two-dimensional array. It features a grid of 9 cells, each containing a number. To the left of the grid, the word 'Row' is written vertically, with a downward-pointing arrow indicating the row indices 0, 1, and 2. Above the grid, the word 'Col' is written horizontally, with a rightward-pointing arrow indicating the column indices 0, 1, and 2. The values in the grid are as follows:

Col →	0	1	2
Row 0	5	10	20
Row 1	25	30	35
Row 2	1	3	4

3. Three-dimensional (3D) array:

Syntax for Declaration of Three Dimensional Array

data_type array_name[sizeof_1st_dimension][sizeof_2nd_dimension][sizeof_3rd_dimension];

where,

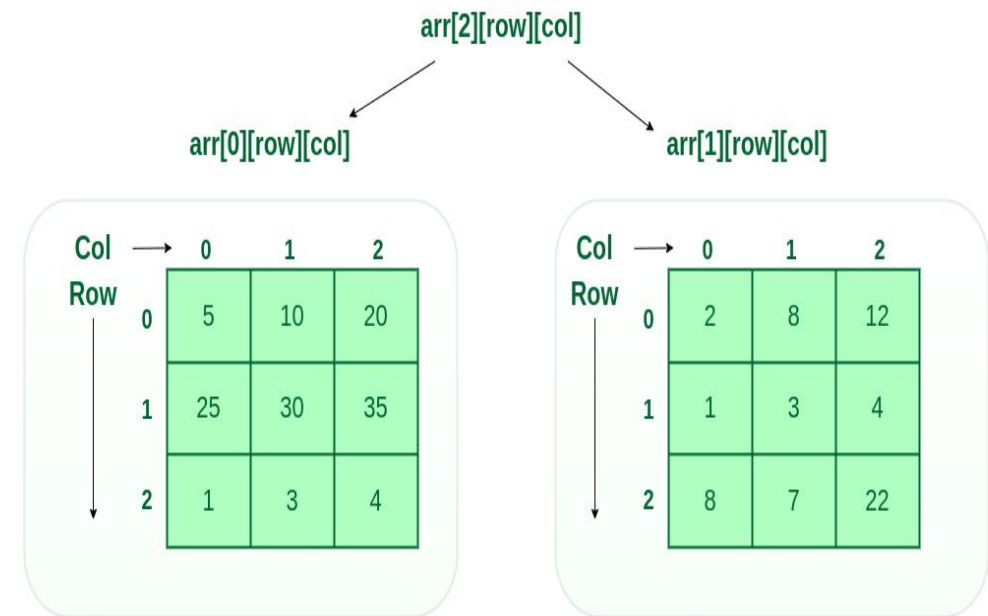
data_type: is a type of data of each array block.

array_name: is the name of the array using which we can refer to it.

sizeof_dimension: is the number of blocks of memory array going to have in the corresponding dimension.

For Example

int nums[5][10][2];



End of Chapter 5